

The Ultimate Maths Reference Guide

Number Theory · Modular Arithmetic · Combinatorics · CRT

Matrix Exponentiation · Geometry · Convex Hull

Sahil Raj (After Dark)

Contents

I	Number Theory	5
1	Primes & Sieves	7
1.1	Sieve of Eratosthenes	7
1.2	Linear Sieve (Smallest Prime Factor)	8
1.3	Primality Testing for Large Numbers	8
1.4	Prime Factorization	9
2	Modular Arithmetic	11
2.1	Core Rules	11
2.2	Fast Modular Exponentiation	11
2.3	Modular Inverse	12
2.4	Inverses for All $i \leq n$ in $\mathcal{O}(n)$	12
2.5	Precomputed Factorials and Binomial Coefficients	12
2.6	Lucas' Theorem (Large n , Small p)	13
3	Combinatorics	15
3.1	Binomial Coefficients — Which Method?	15
3.2	Pascal's Triangle ($n \leq 5000$, Any Modulus)	15
3.3	Factorial Method ($n \leq 2 \times 10^5$, Prime MOD)	16
3.4	Lucas' Theorem (Large n , Small Prime p)	16
3.5	Inclusion-Exclusion Principle	16
3.6	Stars and Bars	17
3.7	Catalan Numbers	18
3.8	Key Combinatorial Identities	18
3.9	Legendre's Formula — Exponent of p in $n!$	19
3.10	Burnside's Lemma	19
3.11	Stirling Numbers & Bell Numbers	20
4	GCD, LCM & Extended Euclidean	23
4.1	GCD and LCM	23
4.2	Extended Euclidean Algorithm	23
4.3	Linear Diophantine Equations	24
5	Chinese Remainder Theorem	25
5.1	The Theorem	25
5.2	Garner's Algorithm (Non-Coprime / Mixed Radix)	26
5.3	Classic CRT Applications	26
6	Euler's Totient & Multiplicative Functions	29
6.1	Euler's Totient Function $\phi(n)$	29
6.2	Euler's Theorem & Fermat's Little Theorem	30
6.3	Multiplicative Functions	30

6.4	Möbius Function & Inclusion-Exclusion	31
7	Matrix Exponentiation	33
7.1	The Core Idea	33
7.2	Matrix Implementation	33
7.3	Fibonacci in $\mathcal{O}(\log n)$	34
7.4	General Linear Recurrences	34
7.5	Graph Path Counting	35
8	Discrete Logarithm & Primitive Roots	37
8.1	Primitive Roots	37
8.2	Baby-step Giant-step (BSGS)	38
9	Advanced Number Theory	41
9.1	Floor Sum — $\sum \lfloor (ai + b)/m \rfloor$	41
9.2	Quadratic Residues & Tonelli–Shanks	42
9.3	Pollard’s Rho — Factorization of Large n	43
10	Generating Functions	45
10.1	Ordinary Generating Functions (OGF)	45
10.2	Exponential Generating Functions (EGF)	46
10.3	Standard GF Tricks	46
II	Geometry	49
11	Geometry Fundamentals	51
11.1	Points, Vectors & Cross Product	51
11.2	Line Intersection	52
11.3	Polygon Area & Point in Polygon	52
11.4	Distance Formulas	53
12	Convex Hull & Sweep Line	55
12.1	Convex Hull — Andrew’s Monotone Chain	55
12.2	Convex Hull Trick (CHT) — DP Optimization	56
12.3	Rotating Calipers — Diameter of a Convex Polygon	57
13	Circles & Circle Inversion	59
13.1	Circle Struct & Basics	59
13.2	Circle–Line Intersection	59
13.3	Circle–Circle Intersection	60
13.4	Circumcircle of 3 Points	60
13.5	Circle Inversion	61
14	Advanced Geometry	63
14.1	Polar Angle Sort	63
14.2	Half-Plane Intersection	63
14.3	Closest Pair of Points	65
14.4	Minkowski Sum of Convex Polygons	66

Part I

Number Theory

Chapter 1

Primes & Sieves

1.1 Sieve of Eratosthenes

How it Works

Mark all multiples of every prime p starting from p^2 as composite. After the sieve, `is_prime[i] = true` iff i is prime.

Why start at p^2 ? Any composite multiple of p smaller than p^2 has a prime factor $< p$, which already sifted it out.

```
1 // O(n log log n) --- the fastest simple sieve
2 vector<bool> sieve(int n) {
3     vector<bool> is_prime(n + 1, true);
4     is_prime[0] = is_prime[1] = false;
5     for (int i = 2; (long long)i * i <= n; i++)
6         if (is_prime[i])
7             for (int j = i * i; j <= n; j += i)
8                 is_prime[j] = false;
9     return is_prime;
10 }
11
12 // Collect primes into a vector
13 vector<int> getPrimes(int n) {
14     auto ip = sieve(n);
15     vector<int> primes;
16     for (int i = 2; i <= n; i++)
17         if (ip[i]) primes.push_back(i);
18     return primes;
19 }
```

Complexity

Time: $\mathcal{O}(n \log \log n)$. Space: $\mathcal{O}(n)$ (use `vector<bool>` for $8\times$ compression).

When to Use

Use when you need to: check primality for many numbers up to N , enumerate all primes, or precompute smallest prime factors. For a single large number use Miller-Rabin instead.

1.2 Linear Sieve (Smallest Prime Factor)

How it Works

The **linear sieve** runs in $\mathcal{O}(n)$ by ensuring each composite is crossed out **exactly once** — by its smallest prime factor (SPF). It also gives you `spf[i]` for free, enabling $\mathcal{O}(\log n)$ factorization of any $i \leq n$.

```

1  const int MAXN = 1e6 + 5;
2  int spf[MAXN];           // spf[i] = smallest prime factor of i
3  vector<int> primes;
4
5  void linearSieve(int n) {
6      fill(spf, spf + n + 1, 0);
7      for (int i = 2; i <= n; i++) {
8          if (!spf[i]) {           // i is prime
9              spf[i] = i;
10             primes.push_back(i);
11         }
12         for (int j = 0; j < (int)primes.size() && primes[j] <= spf[i]
13             && (long long)i * primes[j] <= n; j++) {
14             spf[i * primes[j]] = primes[j];
15         }
16     }
17 }
18
19 // Factorize x in O(log x) using precomputed spf
20 vector<pair<int,int>> factorize(int x) {
21     vector<pair<int,int>> res;
22     while (x > 1) {
23         int p = spf[x], cnt = 0;
24         while (x % p == 0) { x /= p; cnt++; }
25         res.push_back({p, cnt});
26     }
27     return res;
28 }

```

Complexity

Time: $\mathcal{O}(n)$ sieve. Factorization per number: $\mathcal{O}(\log n)$.

Key Insight / Optimization

Multiplicative function trick: If f is multiplicative (e.g. Euler's ϕ , number of divisors $d(n)$, sum of divisors $\sigma(n)$), you can compute $f(n)$ for all $n \leq N$ in $\mathcal{O}(N)$ alongside the linear sieve by tracking f at each prime power.

1.3 Primality Testing for Large Numbers

How it Works

For a single large number n (up to 10^{18}), trial division up to $\sqrt{n}$ runs in $\mathcal{O}(\sqrt{n})$ which is fine up to $n \approx 10^{12}$. For larger n , use **Miller-Rabin**.

```

1  // Trial division --- O(sqrt(n)), good up to ~1e12
2  bool isPrime(long long n) {
3      if (n < 2) return false;
4      if (n == 2 || n == 3) return true;
5      if (n % 2 == 0 || n % 3 == 0) return false;
6      for (long long i = 5; i * i <= n; i += 6)
7          if (n % i == 0 || n % (i + 2) == 0) return false;
8      return true;

```



```

9 }

1 // Deterministic Miller-Rabin for  $n < 3.3e24$ 
2 // Using witnesses {2,3,5,7,11,13,17,19,23,29,31,37} covers all  $n < 3.3e24$ .
3 using ll = long long;
4 using ull = unsigned long long;
5
6 ull mulmod(ull a, ull b, ull m) {
7     return (__uint128_t)a * b % m;
8 }
9 ull powmod(ull a, ull b, ull m) {
10    ull res = 1; a %= m;
11    for (; b; b >>= 1) {
12        if (b & 1) res = mulmod(res, a, m);
13        a = mulmod(a, a, m);
14    }
15    return res;
16 }
17 bool millerRabin(ull n, ull a) {
18     if (n % a == 0) return n == a;
19     ull d = n - 1; int r = 0;
20     while (d % 2 == 0) { d /= 2; r++; }
21     ull x = powmod(a, d, n);
22     if (x == 1 || x == n - 1) return true;
23     for (int i = 0; i < r - 1; i++) {
24         x = mulmod(x, x, n);
25         if (x == n - 1) return true;
26     }
27     return false;
28 }
29 bool isPrimeLarge(ull n) {
30     if (n < 2) return false;
31     for (ull a : {2,3,5,7,11,13,17,19,23,29,31,37})
32         if (!millerRabin(n, a)) return false;
33     return true;
34 }

```

Complexity

Miller-Rabin: $\mathcal{O}(k \log^2 n)$ with k witnesses. Deterministic for $n < 3.3 \times 10^{24}$ with 12 witnesses.

1.4 Prime Factorization

```

1 // Factorize  $n$  in  $\mathcal{O}(\sqrt{n})$  --- works for any  $n$  up to  $\sim 1e12$ 
2 vector<pair<long long,int>> factorize(long long n) {
3     vector<pair<long long,int>> res;
4     for (long long p = 2; p * p <= n; p++) {
5         if (n % p == 0) {
6             int cnt = 0;
7             while (n % p == 0) { n /= p; cnt++; }
8             res.push_back({p, cnt});
9         }
10    }
11    if (n > 1) res.push_back({n, 1});
12    return res;
13 }
14
15 // Number of divisors from factorization:  $\text{prod}(e_i + 1)$ 
16 long long numDivisors(long long n) {
17     long long res = 1;
18     for (auto [p, e] : factorize(n)) res *= (e + 1);
19     return res;
20 }

```

```

21
22 // Sum of divisors: prod( (p^(e+1) - 1) / (p - 1) )
23 long long sumDivisors(long long n) {
24     long long res = 1;
25     for (auto [p, e] : factorize(n)) {
26         long long term = 1, pk = 1;
27         for (int i = 0; i <= e; i++) { term += pk; pk *= p; }
28         res *= term;
29     }
30     return res;
31 }

```

Common Trap / Gotcha

When factorizing in a loop from $p = 2$ to $\sqrt{n}$, always check $n > 1$ after the loop — the remaining n is a prime factor $> \sqrt{n}$ if it wasn't fully divided out.

Classic Problem

Count integers in $[1, N]$ with exactly k prime factors (counted with multiplicity).
 Precompute $\Omega(i)$ (total prime factor count) alongside the linear sieve: $\Omega(i) = \Omega(i/\text{spf}[i]) + 1$.

Chapter 2

Modular Arithmetic

2.1 Core Rules

How it Works

All standard arithmetic operations respect the modulus:

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$$

Division requires the modular inverse: $(a/b) \bmod m = a \cdot b^{-1} \bmod m$, and b^{-1} exists only when $\gcd(b, m) = 1$.

Common Trap / Gotcha

Never use `(%m + m) % m` and forget it. Intermediate products can overflow `long long` ($\approx 9.2 \times 10^{18}$) when $a, b \approx 10^9$ and $m \approx 10^9$: $a \cdot b \approx 10^{18}$ is fine, but $(a \bmod m) \cdot (b \bmod m)$ can reach $(10^9)^2 = 10^{18}$, which is right at the edge. Always cast to `long long` or `__int128` when multiplying.

2.2 Fast Modular Exponentiation

How it Works

Compute $a^b \bmod m$ in $\mathcal{O}(\log b)$ by repeated squaring.

Key recurrence: $a^b = \begin{cases} (a^{b/2})^2 & b \text{ even} \\ a \cdot a^{b-1} & b \text{ odd} \end{cases}$

```
1 // a^b mod m in O(log b)
2 long long power(long long a, long long b, long long m) {
3     long long res = 1;
4     a %= m;
5     while (b > 0) {
6         if (b & 1) res = res * a % m;
7         a = a * a % m;
8         b >>= 1;
9     }
10    return res;
11 }
```

Common Trap / Gotcha

Handle edge cases: $\text{power}(0, 0, m) = 1$ by convention. $\text{power}(a, 0, m) = 1$ for any a . $\text{power}(0, b, m) = 0$ for $b > 0$. Also: if $a = 0$ and $b = 0$, most contest problems define $0^0 = 1$.

2.3 Modular Inverse**How it Works**

The **modular inverse** of a modulo m is the value a^{-1} such that $a \cdot a^{-1} \equiv 1 \pmod{m}$.

Method 1 (Fermat's Little Theorem): When m is prime, $a^{-1} \equiv a^{m-2} \pmod{m}$.

Method 2 (Extended GCD): Works for any m with $\gcd(a, m) = 1$ — see Chapter 3.

```

1 // Modular inverse via Fermat's little theorem --- m must be prime
2 long long modInverse(long long a, long long m) {
3     return power(a, m - 2, m);
4 }
5
6 // Safe modular division
7 long long modDiv(long long a, long long b, long long m) {
8     return a % m * modInverse(b, m) % m;
9 }

```

Complexity

Fermat-based inverse: $\mathcal{O}(\log m)$. Extended GCD-based inverse: $\mathcal{O}(\log \min(a, m))$.

2.4 Inverses for All $i \leq n$ in $\mathcal{O}(n)$ **How it Works**

Computing $i^{-1} \pmod{p}$ for all i from 1 to n individually costs $\mathcal{O}(n \log p)$. The following linear recurrence does it in $\mathcal{O}(n)$:

$$i^{-1} \equiv -\lfloor m/i \rfloor \cdot (m \bmod i)^{-1} \pmod{m}$$

```

1 // Precompute modular inverses of 1..n in O(n), m must be prime
2 vector<long long> allInverses(int n, long long m) {
3     vector<long long> inv(n + 1);
4     inv[1] = 1;
5     for (int i = 2; i <= n; i++)
6         inv[i] = (m - (m / i) * inv[m % i] % m) % m;
7     return inv;
8 }

```

Key Insight / Optimization

Why this works: Write $m = \lfloor m/i \rfloor \cdot i + (m \bmod i)$. Rearranging mod m : $0 \equiv \lfloor m/i \rfloor \cdot i + (m \bmod i) \pmod{m}$. Multiply both sides by $i^{-1} \cdot (m \bmod i)^{-1}$ and simplify. This expresses i^{-1} entirely in terms of $(m \bmod i)^{-1}$, which was already computed since $m \bmod i < i$.

2.5 Precomputed Factorials and Binomial Coefficients

```

1 const long long MOD = 1e9 + 7;
2 const int MAXN = 2e5 + 5;

```

```

3 long long fact[MAXN], inv_fact[MAXN];
4
5 void precompute(int n = MAXN - 1) {
6     fact[0] = 1;
7     for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
8     inv_fact[n] = power(fact[n], MOD - 2, MOD);
9     for (int i = n - 1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD;
10 }
11
12 long long C(int n, int r) {
13     if (r < 0 || r > n) return 0;
14     return fact[n] % MOD * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
15 }
16
17 // Permutation P(n,r) = n! / (n-r)!
18 long long P(int n, int r) {
19     if (r < 0 || r > n) return 0;
20     return fact[n] % MOD * inv_fact[n-r] % MOD;
21 }

```

Complexity

Precomputation: $\mathcal{O}(n)$. Each $\binom{n}{r}$ query: $\mathcal{O}(1)$.

Classic Problem

Stars and Bars: The number of ways to distribute k identical balls into n distinct bins (allowing empty bins) is $\binom{n+k-1}{k}$.

Catalan Number: $C_n = \binom{2n}{n} / (n+1)$ counts valid bracket sequences of length $2n$, BSTs with n nodes, and many other combinatorial structures.

2.6 Lucas' Theorem (Large n , Small p)

How it Works

When p is a small prime and n, r can be very large (not precomputable), use **Lucas' Theorem**:

$$\binom{n}{r} \bmod p = \prod_i \binom{n_i}{r_i} \bmod p$$

where n_i, r_i are the digits of n and r in base p . If any $r_i > n_i$, the result is 0.

```

1 // C(n, r) mod p using Lucas' theorem --- p must be prime, p is small
2 long long lucas(long long n, long long r, long long p) {
3     if (r == 0) return 1;
4     return C(n % p, r % p) * lucas(n / p, r / p, p) % p;
5     // assumes C() is precomputed for small values up to p
6 }

```

Common Trap / Gotcha

Lucas' theorem requires p to be prime. For composite moduli, use the generalized Lucas theorem (Granville's extension) which is significantly more complex.

Chapter 3

Combinatorics

3.1 Binomial Coefficients — Which Method?

When to Use

Constraint	Method	Complexity
$n \leq 5000$, any MOD (even composite)	Pascal's triangle DP	$\mathcal{O}(n^2)$ build, $\mathcal{O}(1)$ query
$n \leq 2 \times 10^5$, MOD prime	Factorial + inverse	$\mathcal{O}(n)$ build, $\mathcal{O}(1)$ query
n huge, p small prime	Lucas' theorem	$\mathcal{O}(p^2 + \log_p n)$
n huge, MOD composite	Granville / CRT	complex

3.2 Pascal's Triangle ($n \leq 5000$, Any Modulus)

How it Works

Build a 2D table using the recurrence $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$, with $\binom{n}{0} = \binom{n}{n} = 1$. **No modular inverse required** — only addition — so it works for **any modulus**, including composite numbers like $10^9 + 6$ or 10^6 .

```
1 // Pascal's triangle: C[n][k] = C(n, k) mod MOD
2 // Works for ANY modulus. Build once, query in O(1).
3 // Memory: O(n^2). Only use when n <= ~5000.
4 const int MAXN = 5001;
5 long long C[MAXN][MAXN];
6
7 void buildPascal(long long MOD) {
8     for (int n = 0; n < MAXN; n++) {
9         C[n][0] = 1;
10        for (int k = 1; k <= n; k++)
11            C[n][k] = (C[n-1][k-1] + C[n-1][k]) % MOD;
12    }
13 }
14
15 // Query: C[n][k] (call buildPascal first)
```

Common Trap / Gotcha

When MOD is composite, the factorial method BREAKS. For example, $\binom{10}{5} \bmod 6$: $\text{fact}[10]^{-1} \bmod 6$ doesn't exist because $\text{gcd}(10!, 6) \neq 1$. Always use Pascal's triangle for composite moduli.

3.3 Factorial Method ($n \leq 2 \times 10^5$, Prime MOD)

```

1 // Precompute:  $O(n)$ . Query  $C(n, r)$ :  $O(1)$ . MOD must be prime.
2 const long long MOD = 1e9 + 7;
3 const int MAXN = 2e5 + 5;
4 long long fact[MAXN], inv_fact[MAXN];
5
6 long long power(long long a, long long b, long long m = MOD) {
7     long long res = 1; a %= m;
8     for (; b >= 1; b >>= 1) { if (b & 1) res = res * a % m; a = a * a % m; }
9     return res;
10 }
11
12 void precompute(int n = MAXN - 1) {
13     fact[0] = 1;
14     for (int i = 1; i <= n; i++) fact[i] = fact[i-1] * i % MOD;
15     inv_fact[n] = power(fact[n], MOD - 2);
16     for (int i = n-1; i >= 0; i--) inv_fact[i] = inv_fact[i+1] * (i+1) % MOD;
17 }
18
19 long long C(int n, int r) {
20     if (r < 0 || r > n) return 0;
21     return fact[n] % MOD * inv_fact[r] % MOD * inv_fact[n-r] % MOD;
22 }
23
24 long long P(int n, int r) { // Permutation  $P(n, r) = n!/(n-r)!$ 
25     if (r < 0 || r > n) return 0;
26     return fact[n] % MOD * inv_fact[n-r] % MOD;
27 }

```

3.4 Lucas' Theorem (Large n , Small Prime p)

```

1 //  $C(n, r) \bmod p$  using Lucas' theorem.  $p$  must be prime and small.
2 // Precompute fact[] up to  $p$  first (call precompute( $p-1$ ) with MOD= $p$ ).
3 long long lucas(long long n, long long r, long long p) {
4     if (r == 0) return 1;
5     return C(n % p, r % p) * lucas(n / p, r / p, p) % p;
6 }

```

3.5 Inclusion-Exclusion Principle

How it Works

For sets $A_1, A_2, \dots, A_k$:

$$\left| \bigcup_{i=1}^k A_i \right| = \sum_i |A_i| - \sum_{i < j} |A_i \cap A_j| + \sum_{i < j < k} |A_i \cap A_j \cap A_k| - \dots$$

In code: iterate over all 2^k subsets of the constraints. If the subset has odd size, add its contribution; if even, subtract.

```

1 // Count integers in  $[1, N]$  divisible by at least one of the given primes.
2 // Inclusion-exclusion over  $2^k$  subsets.
3 long long countDivisible(long long N, vector<long long>& primes) {
4     int k = primes.size();
5     long long ans = 0;
6     for (int mask = 1; mask < (1 << k); mask++) {
7         long long prod = 1;
8         int bits = __builtin_popcount(mask);
9         for (int i = 0; i < k; i++)

```



```

10         if (mask >> i & 1) prod *= primes[i];
11         if (prod > N) continue;
12         if (bits % 2 == 1) ans += N / prod;
13         else               ans -= N / prod;
14     }
15     return ans;
16 }
17
18 // Derangements D(n): permutations of n with no fixed point.
19 // D(n) = (n-1) * (D(n-1) + D(n-2)), D(0)=1, D(1)=0
20 // Alternatively: D(n) = n! * sum_{k=0}^n (-1)^k / k! (inclusion-exclusion)
21 long long derangements(int n, long long MOD) {
22     // Precompute fact[] and inv_fact[] first
23     long long res = 0;
24     for (int k = 0; k <= n; k++) {
25         long long term = inv_fact[k]; // 1/k! mod MOD
26         if (k % 2 == 0) res = (res + term) % MOD;
27         else           res = (res - term + MOD) % MOD;
28     }
29     return fact[n] % MOD * res % MOD;
30 }

```

Classic Problem

Surjective functions: Number of onto functions from an n -set to a k -set:

$$k! \cdot S(n, k) = \sum_{i=0}^k (-1)^i \binom{k}{i} (k-i)^n$$

where $S(n, k)$ is the Stirling number of the second kind.

At most m per bin: Count arrangements of n balls into k bins with $\leq m$ per bin using inclusion-exclusion on the “overflow” events.

3.6 Stars and Bars

How it Works

- **Weak composition** (bins can be empty): distribute n identical balls into k distinct bins = $\binom{n+k-1}{k-1}$.
- **Strict composition** (each bin ≥ 1): $= \binom{n-1}{k-1}$ (substitute $b'_i = b_i - 1$).
- **Each bin $\geq c$:** substitute $b'_i = b_i - c$, solve for $n - ck$ balls into k bins.
- **Each bin $\leq u$:** use inclusion-exclusion on bins exceeding u .

3.7 Catalan Numbers

How it Works

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1}$$

What C_n counts (all equivalent):

- Valid bracket sequences of length $2n$
- Binary trees with $n+1$ leaves
- Monotone lattice paths from $(0,0)$ to (n,n) that never cross the diagonal
- Ways to triangulate a convex $(n+2)$ -gon
- Stack-sortable permutations of $[1..n]$

Values: $C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14, C_5 = 42, C_6 = 132$.

```

1 // C_n mod MOD (prime) in O(1) after precomputation
2 long long catalan(int n) {
3     return C(2*n, n) % MOD * power(n + 1, MOD - 2) % MOD;
4     // Equivalently: C(2n, n) - C(2n, n+1)
5 }
6
7 // Catalan DP: C_n = sum_{i=0}^{n-1} C_i * C_{n-1-i}
8 vector<long long> catalanTable(int maxN, long long MOD) {
9     vector<long long> cat(maxN + 1, 0);
10    cat[0] = cat[1] = 1;
11    for (int n = 2; n <= maxN; n++)
12        for (int i = 0; i < n; i++)
13            cat[n] = (cat[n] + cat[i] * cat[n-1-i]) % MOD;
14    return cat;
15 }
```

3.8 Key Combinatorial Identities

How it Works

Quick reference for the most commonly used identities:

$$\begin{aligned} \binom{n}{k} &= \binom{n}{n-k} && \text{(symmetry)} \\ \binom{n}{k} &= \binom{n-1}{k-1} + \binom{n-1}{k} && \text{(Pascal's rule)} \\ \sum_{k=0}^n \binom{n}{k} &= 2^n && \text{(total subsets)} \\ \sum_{k=0}^r \binom{n+k}{k} &= \binom{n+r+1}{r} && \text{(hockey stick)} \\ \sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} &= \binom{m+n}{r} && \text{(Vandermonde)} \\ \binom{n}{k} \binom{k}{j} &= \binom{n}{j} \binom{n-j}{k-j} && \text{(absorption)} \end{aligned}$$

Complexity

Pascal's triangle: $\mathcal{O}(n^2)$ time and space. Factorial method: $\mathcal{O}(n)$ precompute, $\mathcal{O}(1)$ per query. Catalan DP: $\mathcal{O}(n^2)$; direct formula: $\mathcal{O}(1)$ with precomputed factorials.

3.9 Legendre's Formula — Exponent of p in $n!$

How it Works

The exponent of a prime p in $n!$ (written $v_p(n!)$) is:

$$v_p(n!) = \sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor = \frac{n - s_p(n)}{p-1}$$

where $s_p(n)$ is the digit sum of n in base p . The sum terminates once $p^k > n$.

Key uses: trailing zeros of $n!$ ($= v_5(n!)$), whether $\binom{n}{k}$ is divisible by p (Kummer's theorem), and exact integer arithmetic with factorials.

```

1 // Exponent of prime p in n! -- O(log_p n)
2 long long legendreExp(long long n, long long p) {
3     long long exp = 0;
4     for (long long pk = p; pk <= n; pk *= p)
5         exp += n / pk;
6     return exp;
7 }
8
9 // Trailing zeros of n! = min(v_2(n!), v_5(n!)) = v_5(n!)
10 long long trailingZeros(long long n) { return legendreExp(n, 5); }
11
12 // Kummer's theorem: v_p(C(m+n, m)) = number of carries when adding m and n in base p
13 // Equivalently: (legendreExp(m+n, p) - legendreExp(m, p) - legendreExp(n, p))
14 long long vp_binom(long long n, long long k, long long p) {
15     return legendreExp(n, p) - legendreExp(k, p) - legendreExp(n-k, p);
16 }
17
18 // C(n, k) is NOT divisible by p iff vp_binom(n, k, p) == 0
19 // i.e., no carries when adding k and (n-k) in base p
20 // This is exactly Lucas' theorem: C(n, k) % p != 0 iff each digit of k <= digit of n in base
    p

```

Classic Problem

Classic: How many zeros does $100!$ end with? Answer: $v_5(100!) = 20 + 4 = 24$.

Classic: For what values of n is $\binom{2n}{n}$ odd? Exactly when n has no carrying in binary — i.e., n is a power of 2 minus 1 (Kummer's theorem with $p = 2$).

3.10 Burnside's Lemma

How it Works

Count the number of **distinct objects under symmetry** (orbits of a group action):

$$|\text{distinct objects}| = \frac{1}{|G|} \sum_{g \in G} |\text{Fix}(g)|$$

where $\text{Fix}(g)$ = number of configurations fixed by symmetry g .

Algorithm: For each symmetry operation g in your group G , count how many colorings/arrangements look the same after applying g . Average over all g .

```

1 // Count distinct necklaces with n beads and k colors under ROTATION only.
2 // Rotations: r = 0, 1, ..., n-1 (rotate by r positions).
3 // Fix(rotate by r) = k^gcd(n, r) colorings (each cycle of the rotation gets one color).

```

```

4 long long countNecklaces(int n, int k, long long MOD) {
5     long long ans = 0;
6     for (int r = 0; r < n; r++)
7         ans = (ans + power(k, __gcd(n, r), MOD)) % MOD;
8     // Divide by n: multiply by modular inverse of n
9     return ans % MOD * power(n, MOD - 2, MOD) % MOD;
10 }
11
12 // With ROTATION + REFLECTION (dihedral group D_n, order 2n):
13 long long countBracelets(int n, int k, long long MOD) {
14     long long ans = 0;
15     // n rotations
16     for (int r = 0; r < n; r++)
17         ans = (ans + power(k, __gcd(n, r), MOD)) % MOD;
18     // n reflections
19     if (n % 2 == 0) {
20         // n/2 axis through pairs of beads: fix k^(n/2 + 1) each
21         ans = (ans + (long long)(n/2) % MOD * power(k, n/2 + 1, MOD)) % MOD;
22         // n/2 axis through midpoints of edges: fix k^(n/2) each
23         ans = (ans + (long long)(n/2) % MOD * power(k, n/2, MOD)) % MOD;
24     } else {
25         // n axes through one bead and opposite midpoint: fix k^((n+1)/2) each
26         ans = (ans + (long long)n % MOD * power(k, (n+1)/2, MOD)) % MOD;
27     }
28     return ans % MOD * power(2LL * n % MOD, MOD - 2, MOD) % MOD;
29 }

```

Key Insight / Optimization

General Burnside template: When the group is not just rotations/reflections, enumerate all group elements, compute $|\text{Fix}(g)|$ for each (usually by counting configurations whose “signature” is invariant under g), sum, and divide by $|G|$.

Complexity

$\mathcal{O}(|G| \cdot T)$ where T is the cost to compute $|\text{Fix}(g)|$ for one group element.

3.11 Stirling Numbers & Bell Numbers

How it Works

Stirling numbers of the second kind $S(n, k)$: ways to partition n labelled elements into k non-empty unlabelled subsets.

$$S(n, k) = k \cdot S(n-1, k) + S(n-1, k-1), \quad S(0, 0) = 1, \quad S(n, 0) = S(0, k) = 0$$

Bell number $B_n = \sum_{k=0}^n S(n, k)$: total number of partitions of n elements.

Explicit formula: $S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$ (inclusion-exclusion on surjections).

```

1 // Stirling numbers S[n][k] for all n, k <= MAXN -- O(n^2)
2 const int MAXN = 505;
3 long long S2[MAXN][MAXN]; // S2[n][k] = S(n, k) mod MOD
4
5 void buildStirling(long long MOD) {
6     S2[0][0] = 1;
7     for (int n = 1; n < MAXN; n++)
8         for (int k = 1; k <= n; k++)
9             S2[n][k] = (k * S2[n-1][k] % MOD + S2[n-1][k-1]) % MOD;
10 }

```

```

11 // Bell numbers B[0..n] via Bell triangle -- O(n^2)
12 vector<long long> bellNumbers(int n, long long MOD) {
13     vector<vector<long long>> tri(n+1, vector<long long>(n+1, 0));
14     tri[0][0] = 1;
15     for (int i = 1; i <= n; i++) {
16         tri[i][0] = tri[i-1][i-1];
17         for (int j = 1; j <= i; j++)
18             tri[i][j] = (tri[i][j-1] + tri[i-1][j-1]) % MOD;
19     }
20     vector<long long> B(n+1);
21     for (int i = 0; i <= n; i++) B[i] = tri[i][0];
22     return B;
23 }
24
25 // Number of surjections from n-set to k-set = k! * S(n, k)
26 // = sum_{j=0}^k (-1)^j * C(k, j) * (k-j)^n
27 long long surjections(int n, int k, long long MOD) {
28     long long ans = 0;
29     for (int j = 0; j <= k; j++) {
30         long long term = C(k, j) * power(k - j, n, MOD) % MOD;
31         if (j % 2 == 0) ans = (ans + term) % MOD;
32         else ans = (ans - term + MOD) % MOD;
33     }
34     return ans;
35 }
36

```

Complexity

Stirling triangle: $\mathcal{O}(n^2)$ build. Bell numbers: $\mathcal{O}(n^2)$. Surjection formula: $\mathcal{O}(k \log n)$ with precomputed $C(k, j)$.

Chapter 4

GCD, LCM & Extended Euclidean

4.1 GCD and LCM

How it Works

Euclidean Algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, with $\text{gcd}(a, 0) = a$.

LCM: $\text{lcm}(a, b) = a / \text{gcd}(a, b) \cdot b$ (divide first to avoid overflow).

Why it terminates: Each step strictly reduces $\max(a, b)$; the remainder is always smaller than b .

```
1 long long gcd(long long a, long long b) { return b ? gcd(b, a % b) : a; }
2 long long lcm(long long a, long long b) { return a / gcd(a, b) * b; } // divide first!
3
4 // C++17: std::gcd and std::lcm are in <numeric>
5 #include <numeric>
6 auto g = std::gcd(12LL, 8LL); // 4
7 auto l = std::lcm(4LL, 6LL); // 12
```

Complexity

$\mathcal{O}(\log \min(a, b))$ per call (Fibonacci numbers are the worst case, giving exactly $\log_\phi(\min(a, b))$ steps).

Common Trap / Gotcha

LCM overflow: $\text{lcm}(10^9, 10^9 - 1)$ fits in `long long` ($< 10^{18}$), but `lcm` of three or more numbers can overflow. Compute pairwise and check.

4.2 Extended Euclidean Algorithm

How it Works

The Extended Euclidean Algorithm finds integers x, y (called **Bézout coefficients**) such that:

$$ax + by = \text{gcd}(a, b)$$

This immediately gives a modular inverse: if $\text{gcd}(a, m) = 1$, then $ax + my = 1 \Rightarrow ax \equiv 1 \pmod{m}$, so $x \bmod m$ is the inverse of a .

```
1 // Returns gcd(a, b). Sets x, y such that a*x + b*y = gcd(a, b).
2 long long extGcd(long long a, long long b, long long &x, long long &y) {
3     if (b == 0) { x = 1; y = 0; return a; }
4     long long x1, y1;
5     long long g = extGcd(b, a % b, x1, y1);
```

```

6     x = y1;
7     y = x1 - (a / b) * y1;
8     return g;
9 }
10
11 // Modular inverse using extended GCD --- works for any modulus m (not just prime)
12 // Returns -1 if inverse doesn't exist (gcd(a,m) != 1)
13 long long modInverseExt(long long a, long long m) {
14     long long x, y;
15     long long g = extGcd(a, m, x, y);
16     if (g != 1) return -1;           // no inverse exists
17     return (x % m + m) % m;
18 }

```

Key Insight / Optimization

When to use Extended GCD over Fermat:

- Modulus is **not prime** (Fermat's little theorem doesn't apply).
- You need the actual Bézout coefficients x, y (e.g., for solving linear Diophantine equations).
- The modulus is large and you want to avoid the extra log factor of computing a^{m-2} .

4.3 Linear Diophantine Equations

How it Works

The equation $ax + by = c$ has integer solutions **if and only if** $\gcd(a, b) \mid c$.

If a solution (x_0, y_0) exists, the general solution is:

$$x = x_0 + \frac{b}{g} \cdot t, \quad y = y_0 - \frac{a}{g} \cdot t \quad \text{for any integer } t$$

where $g = \gcd(a, b)$.

```

1 // Solves ax + by = c. Returns false if no solution exists.
2 // If true, sets x, y to one particular solution.
3 bool solveDiophantine(long long a, long long b, long long c,
4                       long long &x, long long &y) {
5     long long g = extGcd(a, b, x, y);
6     if (c % g != 0) return false;
7     long long scale = c / g;
8     x *= scale;
9     y *= scale;
10    return true;
11 }

```

Classic Problem

Classic: You have coins of denominations a and b (coprime). What is the largest amount you *cannot* make? Answer: $ab - a - b$ (the Frobenius/Chicken McNugget theorem).

General: Find the number of non-negative integer solutions (x, y) to $ax + by = c$.

Common Trap / Gotcha

The Bézout coefficients x, y returned by Extended GCD satisfy $|x| \leq b/g$ and $|y| \leq a/g$, so they never overflow if $a, b \leq 10^{18}$ (but watch intermediate products).

Chapter 5

Chinese Remainder Theorem

5.1 The Theorem

How it Works

Chinese Remainder Theorem (CRT): Given a system of congruences with **pairwise coprime** moduli $m_1, m_2, \dots, m_k$:

$$x \equiv r_1 \pmod{m_1}, \quad x \equiv r_2 \pmod{m_2}, \quad \dots, \quad x \equiv r_k \pmod{m_k}$$

There exists a **unique** solution x modulo $M = m_1 \cdot m_2 \cdots m_k$.

Construction: For each i , let $M_i = M/m_i$ and $y_i = M_i^{-1} \pmod{m_i}$. Then:

$$x = \left(\sum_{i=1}^k r_i \cdot M_i \cdot y_i \right) \pmod{M}$$

```
1 // CRT for two congruences: x = r1 (mod m1), x = r2 (mod m2)
2 // m1 and m2 must be coprime. Returns {x, m1*m2}.
3 pair<long long, long long> crt2(long long r1, long long m1,
4                                long long r2, long long m2) {
5     long long x, y;
6     extGcd(m1, m2, x, y);           // m1*x + m2*y = 1
7     long long M = m1 * m2;
8     long long ans = (r1 + m1 * ((r2 - r1) % m2 * x % m2)) % M;
9     return {(ans + M) % M, M};
10 }
11
12 // CRT for a vector of (remainder, modulus) pairs --- all moduli pairwise coprime
13 long long crt(vector<pair<long long, long long>>& congruences) {
14     long long r = 0, M = 1;
15     for (auto [ri, mi] : congruences) {
16         auto [new_r, new_M] = crt2(r, M, ri, mi);
17         r = new_r;
18         M = new_M;
19     }
20     return r;
21 }
```

Complexity

$\mathcal{O}(k \log M)$ where k is the number of congruences and $\log M$ comes from the Extended GCD calls.

5.2 Garner's Algorithm (Non-Coprime / Mixed Radix)

How it Works

When moduli are **not pairwise coprime**, the standard CRT fails. **Garner's Algorithm** reconstructs x in **mixed-radix** form:

$$x = a_1 + a_2 m_1 + a_3 m_1 m_2 + \cdots + a_k m_1 m_2 \cdots m_{k-1}$$

This is especially useful when you want to reconstruct a large number modulo some prime p from its residues modulo several small primes.

```

1 // Garner's algorithm: reconstruct x mod p given x mod m[i] = r[i].
2 // All m[i] are pairwise coprime. Returns x mod p.
3 long long garner(vector<long long> r, vector<long long> m, long long p) {
4     int k = r.size();
5     // coefficients[i] = mixed-radix digit a[i]
6     vector<long long> coeff(k);
7     for (int i = 0; i < k; i++) {
8         coeff[i] = r[i];
9         for (int j = 0; j < i; j++) {
10             // coeff[i] = (coeff[i] - coeff[j]) * inv(m[j], m[i])
11             coeff[i] = (coeff[i] - coeff[j]) % m[i];
12             coeff[i] = coeff[i] * modInverseExt(m[j], m[i]) % m[i];
13             coeff[i] = (coeff[i] + m[i]) % m[i];
14         }
15     }
16     long long ans = 0, prod = 1;
17     for (int i = 0; i < k; i++) {
18         ans = (ans + coeff[i] % p * prod) % p;
19         prod = prod * (m[i] % p) % p;
20     }
21     return ans;
22 }
```

Key Insight / Optimization

Common competitive pattern: Run the same DP/formula with three different NTT-friendly primes (p_1, p_2, p_3) , then use Garner's algorithm to reconstruct the true answer modulo $10^9 + 7$ (or to get the actual integer if it's small enough).

5.3 Classic CRT Applications**Classic Problem**

Problem 1 — Simultaneous congruences: Find the smallest positive x such that $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$.

Apply CRT pairwise. $M = 105$. Answer: $x = 23$.

Classic Problem

Problem 2 — Large integer recovery: You computed a value v that you know satisfies $0 \leq v < 10^{18}$, but all your computations were done modulo $p_1 = 10^9 + 7$ and $p_2 = 10^9 + 9$. Since $p_1 p_2 \approx 10^{18}$, CRT uniquely recovers v .

Common Trap / Gotcha

Overflow in CRT: $m_1 \cdot m_2$ can overflow `long long` when both are $\approx 10^9$. Use `__int128` for the intermediate product, or restructure using modular multiplication.

Chapter 6

Euler's Totient & Multiplicative Functions

6.1 Euler's Totient Function $\phi(n)$

How it Works

$\phi(n)$ counts the integers in $[1, n]$ that are coprime to n .

Formula from factorization:

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

For example: $\phi(12) = 12 \cdot (1 - 1/2) \cdot (1 - 1/3) = 4$.

Key values: $\phi(1) = 1$, $\phi(p) = p - 1$ for prime p , $\phi(p^k) = p^k - p^{k-1}$.

```
1 // phi(n) for a single n in O(sqrt(n))
2 long long phi(long long n) {
3     long long result = n;
4     for (long long p = 2; p * p <= n; p++) {
5         if (n % p == 0) {
6             while (n % p == 0) n /= p;
7             result -= result / p;
8         }
9     }
10    if (n > 1) result -= result / n; // remaining prime factor
11    return result;
12 }
13
14 // Sieve to compute phi(i) for all i in [1, n] in O(n log log n)
15 vector<long long> eulerSieve(int n) {
16     vector<long long> phi(n + 1);
17     iota(phi.begin(), phi.end(), 0); // phi[i] = i initially
18     for (int i = 2; i <= n; i++) {
19         if (phi[i] == i) { // i is prime
20             for (int j = i; j <= n; j += i)
21                 phi[j] -= phi[j] / i;
22         }
23     }
24     return phi;
25 }
```

Complexity

Single $\phi(n)$: $\mathcal{O}(\sqrt{n})$. Sieve for all up to n : $\mathcal{O}(n \log \log n)$. Linear sieve (see Chapter 1): $\mathcal{O}(n)$.

6.2 Euler's Theorem & Fermat's Little Theorem

How it Works**Euler's Theorem:** For $\gcd(a, m) = 1$:

$$a^{\phi(m)} \equiv 1 \pmod{m}$$

Fermat's Little Theorem (special case, m prime):

$$a^{p-1} \equiv 1 \pmod{p} \Rightarrow a^{-1} \equiv a^{p-2} \pmod{p}$$

Consequence for exponent reduction: $a^b \bmod m = a^{b \bmod \phi(m)} \bmod m$ when $\gcd(a, m) = 1$.

```

1 // Compute a^b mod m, with exponent reduction via Euler's theorem
2 // ONLY valid when gcd(a, m) = 1
3 long long powEuler(long long a, long long b, long long m) {
4     long long ph = phi(m);
5     b %= ph;           // exponent reduction
6     return power(a, b, m);
7 }

```

Common Trap / Gotcha**Exponent reduction pitfall:** $a^b \bmod m = a^{b \bmod \phi(m)} \bmod m$ requires $\gcd(a, m) = 1$. If $\gcd(a, m) \neq 1$, you need **extended Euler's theorem**:

$$a^b \equiv a^{\phi(m) + (b \bmod \phi(m))} \pmod{m} \quad \text{for } b \geq \log_2 m$$

This handles the case where a and m share factors.

6.3 Multiplicative Functions

How it WorksA function $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}$ is **multiplicative** if $f(mn) = f(m)f(n)$ whenever $\gcd(m, n) = 1$. Key examples:

- $\phi(n)$: Euler's totient
- $d(n)$: number of divisors
- $\sigma(n)$: sum of divisors
- $\mu(n)$: Möbius function

Multiplicative functions can be computed for all $n \leq N$ in $\mathcal{O}(N)$ using the linear sieve.

```

1 // Compute d(n) (number of divisors) and sigma(n) (sum of divisors)
2 // for all n in [1, N] in O(N log N)
3 void divisorFunctions(int N, vector<long long>& d, vector<long long>& sigma) {
4     d.assign(N + 1, 0);
5     sigma.assign(N + 1, 0);
6     for (int i = 1; i <= N; i++) {
7         for (int j = i; j <= N; j += i) {
8             d[j]++;
9             sigma[j] += i;
10        }
11    }
12 }

```

6.4 Möbius Function & Inclusion-Exclusion

How it Works

The Möbius function $\mu(n)$ is:

$$\mu(n) = \begin{cases} 1 & n = 1 \\ (-1)^k & n = p_1 p_2 \cdots p_k \text{ (squarefree, } k \text{ distinct primes)} \\ 0 & p^2 \mid n \text{ for some prime } p \end{cases}$$

Key identity (Möbius inversion): If $g(n) = \sum_{d|n} f(d)$, then $f(n) = \sum_{d|n} \mu(n/d) g(d)$.

```

1 // Compute mu[i] for all i in [1, n] via sieve
2 vector<int> mobiusSieve(int n) {
3     vector<int> mu(n + 1, 0);
4     vector<bool> is_prime(n + 1, true);
5     vector<int> primes;
6     mu[1] = 1;
7     for (int i = 2; i <= n; i++) {
8         if (is_prime[i]) { primes.push_back(i); mu[i] = -1; }
9         for (int p : primes) {
10             if ((long long)i * p > n) break;
11             is_prime[i * p] = false;
12             if (i % p == 0) { mu[i * p] = 0; break; }
13             mu[i * p] = -mu[i];
14         }
15     }
16     return mu;
17 }
```

Classic Problem

Classic Möbius application: Count pairs (a, b) with $1 \leq a, b \leq N$ such that $\gcd(a, b) = 1$.

Answer: $\sum_{d=1}^N \mu(d) \lfloor N/d \rfloor^2$. This follows from swapping the order of summation with Möbius inversion.

Key Insight / Optimization

GCD sum trick: $\sum_{i=1}^N \sum_{j=1}^N [\gcd(i, j) = 1] = \sum_{d=1}^N \mu(d) \lfloor N/d \rfloor^2$. Combine with **floor division blocking** ($\mathcal{O}(\sqrt{N})$ distinct values of $\lfloor N/d \rfloor$) to compute this in $\mathcal{O}(\sqrt{N})$ after $\mathcal{O}(N)$ prefix sum precomputation of μ .

Chapter 7

Matrix Exponentiation

7.1 The Core Idea

How it Works

If a recurrence can be expressed as $\mathbf{v}_{n+1} = A \cdot \mathbf{v}_n$ for a fixed matrix A , then:

$$\mathbf{v}_n = A^n \cdot \mathbf{v}_0$$

We compute A^n using **matrix fast exponentiation** in $\mathcal{O}(k^3 \log n)$, where k is the matrix dimension.

This converts any **linear recurrence of order k** solvable in $\mathcal{O}(n)$ time into $\mathcal{O}(k^3 \log n)$ — a massive win when $n \leq 10^{18}$.

7.2 Matrix Implementation

```
1  const long long MOD = 1e9 + 7;
2  using Matrix = vector<vector<long long>>;
3
4  Matrix multiply(const Matrix& A, const Matrix& B) {
5      int n = A.size(), m = B[0].size(), p = B.size();
6      Matrix C(n, vector<long long>(m, 0));
7      for (int i = 0; i < n; i++)
8          for (int k = 0; k < p; k++) if (A[i][k])
9              for (int j = 0; j < m; j++)
10                 C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
11     return C;
12 }
13
14 Matrix matpow(Matrix A, long long b) {
15     int n = A.size();
16     // Identity matrix
17     Matrix res(n, vector<long long>(n, 0));
18     for (int i = 0; i < n; i++) res[i][i] = 1;
19     while (b > 0) {
20         if (b & 1) res = multiply(res, A);
21         A = multiply(A, A);
22         b >>= 1;
23     }
24     return res;
25 }
```

7.3 Fibonacci in $\mathcal{O}(\log n)$

How it Works

The Fibonacci recurrence $F_n = F_{n-1} + F_{n-2}$ translates to:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}$$

So F_n is the $(0, 1)$ entry (or $(1, 0)$) of A^n where $A = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$.

```

1 // F(n) mod MOD in O(log n)
2 long long fibonacci(long long n) {
3     if (n == 0) return 0;
4     Matrix A = {{1, 1}, {1, 0}};
5     Matrix res = matpow(A, n);
6     return res[0][1];    // = F(n)
7 }
```

7.4 General Linear Recurrences

How it Works

Given $f(n) = c_1 f(n-1) + c_2 f(n-2) + \dots + c_k f(n-k)$, construct the **companion matrix**:

$$A = \begin{pmatrix} c_1 & c_2 & c_3 & \dots & c_k \\ 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{pmatrix}$$

Then $\mathbf{v}_n = A^{n-k+1} \cdot \mathbf{v}_{k-1}$ where $\mathbf{v}_i = (f(i), f(i-1), \dots, f(i-k+1))^T$.

```

1 // Evaluate a linear recurrence at position n in O(k^3 log n)
2 // coeffs[i] = coefficient of f(n-1-i), base[i] = f(i)
3 long long linearRecurrence(long long n, vector<long long> coeffs,
4                             vector<long long> base) {
5     int k = coeffs.size();
6     if (n < k) return base[n];
7
8     // Build companion matrix
9     Matrix A(k, vector<long long>(k, 0));
10    for (int j = 0; j < k; j++) A[0][j] = coeffs[j] % MOD;
11    for (int i = 1; i < k; i++) A[i][i-1] = 1;
12
13    Matrix res = matpow(A, n - k + 1);
14
15    // Multiply res * base_vector (v_{k-1})
16    long long ans = 0;
17    for (int j = 0; j < k; j++)
18        ans = (ans + res[0][j] % MOD * base[k-1-j]) % MOD;
19    return ans;
20 }
```

Complexity

$\mathcal{O}(k^3 \log n)$ per query. For $k = 2$ (Fibonacci-like): $\mathcal{O}(8 \log n)$. For order-6 recurrences: $\mathcal{O}(216 \log n)$.

7.5 Graph Path Counting

How it Works

If A is the **adjacency matrix** of a graph, then $(A^k)_{ij}$ counts the number of **walks of length exactly k** from vertex i to vertex j .

Similarly, the number of paths of length $\leq k$ can be computed using the matrix $(I + A)^k$ (but be careful — this counts paths of all lengths up to k , not exactly k).

```

1 // Count walks of length exactly k from u to v in a graph with adjacency matrix A
2 long long countWalks(vector<vector<long long>>& A, int u, int v, long long k) {
3     Matrix res = matpow(A, k);
4     return res[u][v];
5 }
```

Classic Problem

Classic: Count the number of ways to tile a $2 \times n$ board with 1×2 dominoes. The recurrence $f(n) = f(n-1) + f(n-2)$ (Fibonacci-like) can be evaluated at $n = 10^{18}$ in $\mathcal{O}(\log n)$ using matrix expo.

Harder: Count tilings of a $k \times n$ board — encode row states as bitmasks, giving a $2^k \times 2^k$ transition matrix.

Common Trap / Gotcha

Matrix multiplication order matters. $(AB) \neq (BA)$ in general. When computing $A^n \cdot \mathbf{v}$, multiply left to right. The identity matrix must be square with 1s on the diagonal — never use the zero matrix as the initial `res`.

Key Insight / Optimization

Cayley-Hamilton shortcut: For Fibonacci specifically, a faster closed-form approach uses the identity $F_{2n} = F_n(2F_{n+1} - F_n)$ and $F_{2n+1} = F_n^2 + F_{n+1}^2$, giving $\mathcal{O}(\log n)$ with just scalar arithmetic. But the matrix approach generalizes to *any* linear recurrence, making it the universally applicable tool.

Chapter 8

Discrete Logarithm & Primitive Roots

8.1 Primitive Roots

How it Works

A **primitive root** g modulo a prime p is an element whose powers generate all of $\mathbb{Z}_p^* = \{1, 2, \dots, p-1\}$. Its order is $p-1$.

Finding g : Try $g = 2, 3, 4, \dots$. For each candidate, check that $g^{(p-1)/q} \not\equiv 1 \pmod{p}$ for every prime factor q of $p-1$. The first one that passes is a primitive root.

Key fact: Primitive roots exist for p prime, p^k , $2p^k$, 2, and 4.

```
1 // Find smallest primitive root modulo prime p -- O(sqrt(p) + d(p-1) * log p)
2 long long primitiveRoot(long long p) {
3     // Factorize p-1
4     vector<long long> factors;
5     long long phi = p - 1, n = phi;
6     for (long long i = 2; i * i <= n; i++)
7         if (n % i == 0) { factors.push_back(i); while (n % i == 0) n /= i; }
8     if (n > 1) factors.push_back(n);
9
10    // Try candidates
11    for (long long g = 2; g < p; g++) {
12        bool ok = true;
13        for (long long q : factors)
14            if (power(g, phi / q, p) == 1) { ok = false; break; }
15        if (ok) return g;
16    }
17    return -1; // no primitive root (shouldn't happen for prime p)
18 }
19
20 // If g is a primitive root mod p, then g^0, g^1, ..., g^(p-2) give all of Z_p*.
21 // log_g(x) = discrete logarithm of x base g mod p (solved by BSGS below).
```

Key Insight / Optimization

Use case: NTT-friendly primes like $998244353 = 119 \cdot 2^{23} + 1$ have $g = 3$ as a primitive root. For a DFT of size $n = 2^k \leq 2^{23}$, use $\omega = g^{(p-1)/n} \pmod{p}$ as the principal n -th root of unity.

8.2 Baby-step Giant-step (BSGS)

How it Works

Problem: Find $x \geq 0$ such that $a^x \equiv b \pmod{p}$ (with p prime, $\gcd(a, p) = 1$).

Idea: Write $x = i\sqrt{p} - j$ where $0 \leq i, j \leq \lceil \sqrt{p} \rceil$. Then $a^{i\sqrt{p}} \equiv b \cdot a^j \pmod{p}$.

Baby step: precompute $b \cdot a^j$ for $j = 0 \dots \lceil \sqrt{p} \rceil$ into a hash map.

Giant step: for $i = 1 \dots \lceil \sqrt{p} \rceil$, compute $a^{i\sqrt{p}}$ and look it up.

```

1 // Discrete log: find smallest x >= 0 s.t. a^x = b (mod p), p prime, gcd(a,p)=1.
2 // Returns -1 if no solution.
3 long long BSGS(long long a, long long b, long long p) {
4     a %= p; b %= p;
5     if (b == 1 || a == 0) return b == 1 ? 0 : (a == b ? 1 : -1);
6
7     long long m = (long long)ceil(sqrt((double)p));
8     unordered_map<long long, long long> table;
9
10    // Baby step: store b * a^j for j = 0..m-1
11    long long cur = b;
12    for (long long j = 0; j < m; j++) {
13        table[cur] = j;
14        cur = cur * a % p;
15    }
16
17    // Giant step: compute a^(i*m), look for match
18    long long am = power(a, m, p);
19    cur = am;
20    for (long long i = 1; i <= m; i++) {
21        if (table.count(cur)) {
22            long long x = i * m - table[cur];
23            return x % (p - 1); // normalize: order divides p-1
24        }
25        cur = cur * am % p;
26    }
27    return -1;
28 }
29
30 // Extended BSGS: works even when gcd(a, p) != 1 (non-prime moduli too).
31 // Reduces to the coprime case by dividing out common factors iteratively.
32 long long extBSGS(long long a, long long b, long long p) {
33     a %= p; b %= p;
34     if (b == 1 || p == 1) return 0;
35     long long cnt = 0, t = 1;
36     for (long long g; (g = __gcd(a, p)) != 1; cnt++) {
37         if (b % g != 0) return -1;
38         b /= g; p /= g;
39         t = t * (a / g) % p;
40         if (t == b) return cnt;
41     }
42     // Now gcd(a, p) = 1, use standard BSGS shifted by cnt
43     long long m = (long long)ceil(sqrt((double)p));
44     unordered_map<long long, long long> table;
45     long long cur = b;
46     for (long long j = 0; j < m; j++) { table[cur] = j; cur = cur * a % p; }
47     long long am = power(a, m, p);
48     cur = t * am % p;
49     for (long long i = 1; i <= m + 1; i++) {
50         if (table.count(cur)) return i * m - table[cur] + cnt;
51         cur = cur * am % p;
52     }
53     return -1;
54 }

```

Complexity

Standard BSGS: $\mathcal{O}(\sqrt{p})$ time and space.

Extended BSGS: $\mathcal{O}(\log p \cdot \sqrt{p})$ time, $\mathcal{O}(\sqrt{p})$ space.

Classic Problem

Classic: Solve $2^x \equiv 3 \pmod{10^9 + 7}$.

Classic: Given $g^x \equiv h \pmod{p}$ for known primitive root g , solve for x — this is the Diffie-Hellman discrete log problem.

Pattern: Any equation $f(a^x) \equiv b \pmod{p}$ where f is a simple polynomial often reduces to BSGS after substitution.

Chapter 9

Advanced Number Theory

9.1 Floor Sum — $\sum \lfloor (ai + b)/m \rfloor$

How it Works

Compute $f(a, b, m, n) = \sum_{i=0}^{n-1} \left\lfloor \frac{ai + b}{m} \right\rfloor$ in $\mathcal{O}(\log m)$ using a Euclidean-style reduction (identical to the Euclidean algorithm).

Why it matters: Appears in count of lattice points under a line, number-theory functions involving $\lfloor n/k \rfloor$, and competitive problems up to 2300.

```
1 // Floor sum: sum_{i=0}^{n-1} floor((a*i + b) / m) -- O(log m)
2 // All values non-negative. Uses the same reduction as the Euclidean algorithm.
3 long long floorSum(long long n, long long m, long long a, long long b) {
4     long long ans = 0;
5     // Reduce a and b modulo m, accounting for overflow
6     ans += (a / m) * (n * (n - 1) / 2); a %= m;
7     ans += (b / m) * n; b %= m;
8     long long y_max = (a * n + b) / m;
9     long long x_max = y_max * m - b;
10    if (y_max == 0) return ans;
11    ans += (n - (x_max + a - 1) / a) * y_max;
12    ans += floorSum(y_max, a, m, (a - x_max % a) % a);
13    return ans;
14 }
15
16 // Count lattice points (i, j) with 0 <= i < n, 0 < j <= floor((a*i+b)/m)
17 // = floorSum(n, m, a, b)
```

Classic Problem

Classic: Count pairs (x, y) with $1 \leq x \leq n$, $1 \leq y \leq m$ and $y \leq ax/b$ — reduce to `floorSum`.

Classic: $\sum_{k=1}^n \lfloor n/k \rfloor$ in $\mathcal{O}(\sqrt{n})$ using the fact that $\lfloor n/k \rfloor$ takes only $\mathcal{O}(\sqrt{n})$ distinct values (floor-division blocking).

9.2 Quadratic Residues & Tonelli–Shanks

How it Works

a is a **quadratic residue** mod prime p if $\exists x : x^2 \equiv a \pmod{p}$.

Euler's criterion: $a^{(p-1)/2} \equiv \begin{cases} 1 \pmod{p} & a \text{ is a QR} \\ -1 \pmod{p} & a \text{ is a non-residue} \end{cases}$

Tonelli-Shanks finds the actual square root in $\mathcal{O}(\log^2 p)$.

```

1 // Is a a quadratic residue mod prime p? (Euler's criterion)
2 bool isQR(long long a, long long p) {
3     return power(a, (p - 1) / 2, p) == 1;
4 }
5
6 // Tonelli-Shanks: find x s.t. x^2 = n (mod p). Returns -1 if no solution.
7 long long sqrtMod(long long n, long long p) {
8     n %= p;
9     if (n < 0) n += p;
10    if (n == 0) return 0;
11    if (!isQR(n, p)) return -1;
12    if (p % 4 == 3) return power(n, (p + 1) / 4, p);
13
14    // Factor out 2s from p-1: p-1 = Q * 2^S
15    long long Q = p - 1, S = 0;
16    while (Q % 2 == 0) { Q /= 2; S++; }
17
18    // Find a non-residue z
19    long long z = 2;
20    while (isQR(z, p)) z++;
21
22    long long M = S;
23    long long c = power(z, Q, p);
24    long long t = power(n, Q, p);
25    long long R = power(n, (Q + 1) / 2, p);
26
27    while (true) {
28        if (t == 1) return R;
29        long long i = 1, tmp = t * t % p;
30        while (tmp != 1) { tmp = tmp * tmp % p; i++; }
31        long long b = power(c, power(2, M - i - 1, p - 1), p);
32        M = i;
33        c = b * b % p;
34        t = t * c % p;
35        R = R * b % p;
36    }
37 }

```

Complexity

Euler's criterion: $\mathcal{O}(\log p)$. Tonelli-Shanks: $\mathcal{O}(\log^2 p)$ expected.

Common Trap / Gotcha

When $p \equiv 3 \pmod{4}$, the simpler formula $x = n^{(p+1)/4} \pmod{p}$ works directly. Tonelli-Shanks is only needed for $p \equiv 1 \pmod{4}$.

9.3 Pollard's Rho — Factorization of Large n

How it Works

Pollard's Rho factors n in $\mathcal{O}(n^{1/4})$ expected time using a random walk $x \mapsto x^2 + c \pmod{n}$ and Floyd's cycle detection on the GCD.

Combine with **Miller-Rabin** primality test (Chapter 1) for a complete $\mathcal{O}(n^{1/4} \log n)$ factorizer for $n \leq 10^{18}$.

```

1 // Miller-Rabin + Pollard's Rho complete factorizer for n <= 1e18
2
3 using ull = unsigned long long;
4 using ll  = long long;
5
6 ull mulmod(ull a, ull b, ull m) { return (__uint128_t)a * b % m; }
7 ull powmod(ull a, ull b, ull m) {
8     ull r = 1; for (a %= m; b; b >>= 1) { if (b&1) r=mulmod(r,a,m); a=mulmod(a,a,m); }
9     return r;
10 }
11 bool millerRabin(ull n, ull a) {
12     if (n%a==0) return n==a;
13     ull d=n-1; int r=0;
14     while (d%2==0) { d/=2; r++; }
15     ull x=powmod(a,d,n);
16     if (x==1||x==n-1) return true;
17     for (int i=0;i<r-1;i++) { x=mulmod(x,x,n); if(x==n-1) return true; }
18     return false;
19 }
20 bool isPrime(ull n) {
21     if (n<2) return false;
22     for (ull a : {2,3,5,7,11,13,17,19,23,29,31,37})
23         if (!millerRabin(n,a)) return false;
24     return true;
25 }
26 ull pollardRho(ull n) {
27     if (n%2==0) return 2;
28     static mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
29     while (true) {
30         ull x = rng()%(n-2)+2, c = rng()%(n-1)+1, y=x, d=1;
31         while (d==1) {
32             x = (mulmod(x,x,n)+c)%n;
33             y = (mulmod(y,y,n)+c)%n;
34             y = (mulmod(y,y,n)+c)%n;
35             d = __gcd(x>y?x-y:y-x, n);
36         }
37         if (d!=n) return d;
38     }
39 }
40
41 // Returns all prime factors of n (with multiplicity), sorted
42 vector<ull> factorize(ull n) {
43     if (n==1) return {};
44     if (isPrime(n)) return {n};
45     ull d = pollardRho(n);
46     auto f1 = factorize(d), f2 = factorize(n/d);
47     for (ull x : f2) f1.push_back(x);
48     sort(f1.begin(), f1.end());
49     return f1;
50 }

```

Complexity

Pollard's Rho: $\mathcal{O}(n^{1/4})$ expected per factor found. Full factorization of $n \leq 10^{18}$: typically $< 10^7$ operations.

Chapter 10

Generating Functions

10.1 Ordinary Generating Functions (OGF)

How it Works

An **OGF** represents a sequence $a_0, a_1, a_2, \dots$ as a formal power series:

$$F(x) = \sum_{n \geq 0} a_n x^n$$

Core operations on OGFs:

- **Addition:** $(F + G)(x) = \sum (a_n + b_n)x^n$ — pointwise sum of sequences.
- **Multiplication (convolution):** $(F \cdot G)(x) = \sum_n (\sum_{k=0}^n a_k b_{n-k}) x^n$.
- **Coefficient of x^n** in $\frac{1}{1-x} = 1$, in $\frac{1}{(1-x)^k} = \binom{n+k-1}{k-1}$.

Key OGFs:

$$\frac{1}{1-x} = 1 + x + x^2 + \dots, \quad \frac{1}{1-x^k} = (\text{multiples of } k), \quad (1+x)^n = \sum_k \binom{n}{k} x^k$$

```
1 // Polynomial multiplication (convolution) in O(n*m) -- brute force
2 // For large degrees, use FFT (see FFT guide).
3 vector<long long> polyMul(vector<long long>& A, vector<long long>& B, long long MOD) {
4     int n = A.size(), m = B.size();
5     vector<long long> C(n + m - 1, 0);
6     for (int i = 0; i < n; i++)
7         for (int j = 0; j < m; j++)
8             C[i+j] = (C[i+j] + A[i] * B[j]) % MOD;
9     return C;
10 }
11
12 // Coin change generating function:
13 // Number of ways to make sum n using coins of denominations d[0..k-1] (unlimited)
14 // = coefficient of x^n in product of 1/(1-x^{d[i]})
15 // Computed via DP in O(n * k)
16 vector<long long> coinChangeWays(int n, vector<int>& coins, long long MOD) {
17     vector<long long> dp(n + 1, 0);
18     dp[0] = 1;
19     for (int c : coins)
20         for (int j = c; j <= n; j++)
21             dp[j] = (dp[j] + dp[j - c]) % MOD;
22     return dp;
23 }
```

10.2 Exponential Generating Functions (EGF)

How it Works

An **EGF** represents sequence a_n as:

$$F(x) = \sum_{n \geq 0} a_n \frac{x^n}{n!}$$

Why EGF? When combining *labelled* structures, the product of EGFs gives the EGF of the combined structure (with the $n!$ automatically handling the labelling).

Key EGFs:

$$e^x = \sum_n \frac{x^n}{n!}, \quad e^{kx} = \sum_n k^n \frac{x^n}{n!}, \quad \ln(1+x) = \sum_{n \geq 1} \frac{(-1)^{n-1}}{n} x^n$$

EGF product rule: If F is EGF for labelled structure A and G for B , then $F \cdot G$ is EGF for “ordered pair of A and B on a shared labelled set” (with all ways to split the label set).

```

1 // EGF multiplication: computes coefficients of [x^n/n!] in F(x)*G(x)
2 // Input: A[n] = a_n (NOT a_n/n!). Output: C[n] = c_n s.t. C(x)=A(x)*B(x) as EGFs.
3 // EGF product: c_n = sum_{k=0}^n C(n,k) * a_k * b_{n-k}
4 vector<long long> egfMul(vector<long long>& A, vector<long long>& B,
5                          long long MOD, int maxN) {
6     // Precompute fact, inv_fact first
7     vector<long long> C(maxN + 1, 0);
8     for (int n = 0; n <= maxN; n++)
9         for (int k = 0; k <= n; k++)
10             C[n] = (C[n] + ::C(n, k) % MOD * A[k] % MOD * B[n-k]) % MOD;
11     return C;
12 }
13
14 // Number of ways to arrange n labelled items into k ordered groups (each non-empty)
15 // = surjections * ways to order groups = Stirling(n,k) * k!
16 // EGF of "non-empty set" = e^x - 1, so EGF of k-partition = (e^x-1)^k / k!
17 // Coefficient of x^n/n! in (e^x-1)^k / k! = S(n,k) (Stirling 2nd kind)

```

10.3 Standard GF Tricks

How it Works

Tiling problems: The number of ways to tile a $1 \times n$ strip with tiles of sizes $1, 2, 3, \dots$ has OGF $\frac{1}{1-x-x^2-x^3-\dots} = \frac{1-x}{1-2x}$ for all tile sizes allowed.

Partition function: Number of ways to write n as an *unordered* sum of positive integers has OGF $\prod_{k \geq 1} \frac{1}{1-x^k}$.

GF for linear recurrences: If $a_n = c_1 a_{n-1} + c_2 a_{n-2} + \dots + c_k a_{n-k}$, the OGF is a rational function $\frac{P(x)}{Q(x)}$ where $Q(x) = 1 - c_1 x - c_2 x^2 - \dots - c_k x^k$.

```

1 // Partition numbers p(n): ways to write n as unordered sum of positive integers
2 // Euler's pentagonal theorem: p(n) = sum_{k!=0} (-1)^{k+1} p(n - k(3k-1)/2)
3 // Pentagonal numbers: 1, 2, 5, 7, 12, 15, 22, 26, ... (k=1,-1,2,-2,...)
4 vector<long long> partitionNumbers(int maxN, long long MOD) {
5     vector<long long> p(maxN + 1, 0);
6     p[0] = 1;
7     for (int n = 1; n <= maxN; n++) {
8         for (int k = 1; ; k++) {
9             long long g1 = (long long)k*(3*k-1)/2;
10            long long g2 = (long long)k*(3*k+1)/2;
11            if (g1 > n) break;
12            if (k % 2 == 1) p[n] = (p[n] + p[n-g1]) % MOD;

```

```

13         else                p[n] = (p[n] - p[n-g1] + MOD) % MOD;
14         if (g2 <= n) {
15             if (k % 2 == 1) p[n] = (p[n] + p[n-g2]) % MOD;
16             else          p[n] = (p[n] - p[n-g2] + MOD) % MOD;
17         }
18     }
19 }
20 return p;
21 }

```

Complexity

OGF convolution (brute force): $\mathcal{O}(n^2)$. With FFT/NTT: $\mathcal{O}(n \log n)$ (see separate FFT guide).

EGF multiplication: $\mathcal{O}(n^2)$ brute force, $\mathcal{O}(n \log n)$ with FFT.

Partition numbers (Euler's pentagonal): $\mathcal{O}(n\sqrt{n})$.

Key Insight / Optimization

The GF mindset: When you see a counting problem with a recursive structure (“a valid sequence is a head element followed by a valid tail”), translate it directly to a GF equation. Solving the GF equation (often just algebra) gives you either a closed form or a fast recurrence you wouldn't have found otherwise.

Part II

Geometry

Chapter 11

Geometry Fundamentals

11.1 Points, Vectors & Cross Product

How it Works

Everything in 2D geometry reduces to **vectors** and the **cross product**.

Given vectors $\mathbf{u} = (u_x, u_y)$ and $\mathbf{v} = (v_x, v_y)$:

$$\mathbf{u} \times \mathbf{v} = u_x v_y - u_y v_x$$

- > 0 : $\mathbf{v}$ is **counterclockwise** (left turn) from $\mathbf{u}$
- $= 0$: $\mathbf{u}$ and $\mathbf{v}$ are **collinear**
- < 0 : $\mathbf{v}$ is **clockwise** (right turn) from $\mathbf{u}$

$|\mathbf{u} \times \mathbf{v}|$ equals twice the **area of the triangle** formed by $\mathbf{0}$, $\mathbf{u}$, $\mathbf{v}$.

```
1 using ld = long double;
2 const ld EPS = 1e-9;
3
4 struct P {
5     ld x, y;
6     P(ld x = 0, ld y = 0) : x(x), y(y) {}
7     P operator+(P o) const { return {x + o.x, y + o.y}; }
8     P operator-(P o) const { return {x - o.x, y - o.y}; }
9     P operator*(ld t) const { return {x * t, y * t}; }
10    ld dot(P o) const { return x * o.x + y * o.y; }
11    ld cross(P o) const { return x * o.y - y * o.x; }
12    ld norm() const { return sqrtl(x * x + y * y); }
13    ld norm2() const { return x * x + y * y; } // squared norm, no sqrt
14    bool operator<(P o) const { // lexicographic order
15        return x < o.x - EPS || (fabsl(x - o.x) < EPS && y < o.y - EPS);
16    }
17    bool operator==(P o) const {
18        return fabsl(x - o.x) < EPS && fabsl(y - o.y) < EPS;
19    }
20 };
21
22 // Sign of cross product: +1, 0, or -1
23 int sign(ld v) { return (v > EPS) - (v < -EPS); }
24
25 // Orientation of triangle (a, b, c):
26 // +1 = CCW, 0 = collinear, -1 = CW
27 int orient(P a, P b, P c) { return sign((b - a).cross(c - a)); }
```

Common Trap / Gotcha

Use **long long** for integer geometry. When input coordinates are integers, *never* use floating point for the cross product — use **long long** to avoid precision errors entirely. Switch to **long double** only when you truly need distances or angles.

11.2 Line Intersection

```

1 // Line through points a, b. Returns true if it intersects line through c, d.
2 // Sets 'out' to the intersection point.
3 bool lineIntersect(P a, P b, P c, P d, P &out) {
4     ld d1 = (b - a).cross(d - c);
5     if (fabs1(d1) < EPS) return false;    // parallel or coincident
6     ld t = (c - a).cross(d - c) / d1;
7     out = a + (b - a) * t;
8     return true;
9 }
10
11 // Segment intersection: do segments (a,b) and (c,d) properly intersect?
12 bool segmentsIntersect(P a, P b, P c, P d) {
13     int d1 = orient(c, d, a), d2 = orient(c, d, b);
14     int d3 = orient(a, b, c), d4 = orient(a, b, d);
15     if (d1 * d2 < 0 && d3 * d4 < 0) return true;
16     // Check collinear overlap
17     auto onSeg = [](P p, P a, P b) {
18         return fabs1((p-a).cross(b-a)) < EPS
19             && (p-a).dot(b-a) > -EPS
20             && (p-b).dot(a-b) > -EPS;
21     };
22     return onSeg(a,c,d) || onSeg(b,c,d) || onSeg(c,a,b) || onSeg(d,a,b);
23 }

```

11.3 Polygon Area & Point in Polygon**How it Works**

Shoelace Formula: Area of a polygon with vertices $p_0, p_1, \dots, p_{n-1}$ (in order):

$$\text{Area} = \frac{1}{2} \left| \sum_{i=0}^{n-1} (p_i \times p_{(i+1) \bmod n}) \right|$$

For integer coordinates, multiply by 2 and keep as integer to avoid any floating point.

```

1 // Signed area * 2 (positive = CCW). Use integer type for exact arithmetic.
2 long long area2(vector<P>& poly) {
3     long long res = 0;
4     int n = poly.size();
5     for (int i = 0; i < n; i++)
6         res += poly[i].cross(poly[(i+1) % n]);
7     return res;    // divide by 2 for actual area
8 }
9
10 // Point in polygon: winding number algorithm. Returns true if p is inside.
11 // Works correctly for non-convex polygons. O(n).
12 bool pointInPolygon(P p, vector<P>& poly) {
13     int winding = 0, n = poly.size();
14     for (int i = 0; i < n; i++) {
15         P a = poly[i], b = poly[(i+1)%n];
16         if (fabs1((a-p).cross(b-p)) < EPS && (a-p).dot(b-p) < EPS)
17             return true;    // on boundary

```

```

18     if (a.y <= p.y) {
19         if (b.y > p.y && orient(a, b, p) > 0) winding++;
20     } else {
21         if (b.y <= p.y && orient(a, b, p) < 0) winding--;
22     }
23 }
24 return winding != 0;
25 }

```

Complexity

Shoelace: $\mathcal{O}(n)$. Point-in-polygon (winding number): $\mathcal{O}(n)$. For convex polygons, binary search gives $\mathcal{O}(\log n)$ point-in-polygon.

11.4 Distance Formulas

```

1  // Distance from point p to line through (a, b)
2  ld pointToLine(P p, P a, P b) {
3      return fabs1((p - a).cross(b - a)) / (b - a).norm();
4  }
5
6  // Distance from point p to segment (a, b)
7  ld pointToSegment(P p, P a, P b) {
8      if ((b - a).dot(p - a) < EPS) return (p - a).norm(); // past a
9      if ((a - b).dot(p - b) < EPS) return (p - b).norm(); // past b
10     return pointToLine(p, a, b);
11 }
12
13 // Distance between two parallel/non-parallel segments: use pointToSegment
14 ld segmentToSegment(P a, P b, P c, P d) {
15     if (segmentsIntersect(a, b, c, d)) return 0;
16     return min({pointToSegment(a,c,d), pointToSegment(b,c,d),
17                pointToSegment(c,a,b), pointToSegment(d,a,b)});
18 }

```

Key Insight / Optimization

Squared distance avoids sqrt. When comparing distances, always compare squared distances (`norm2()`) to avoid floating-point error and save computation. Only take `sqrt` when you need the actual value.

Chapter 12

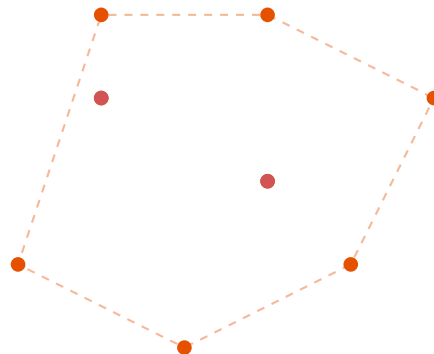
Convex Hull & Sweep Line

12.1 Convex Hull — Andrew's Monotone Chain

How it Works

A **convex hull** is the smallest convex polygon containing all points. Andrew's Monotone Chain builds it in $\mathcal{O}(n \log n)$ by separately constructing the lower and upper hull.

Key step: While adding a new point makes a **right turn** (or goes straight), pop the last hull point. Only left turns are kept.



Hull (blue) vs interior (red)

```
1 // Returns vertices of convex hull in CCW order.
2 // Collinear points on the hull boundary are excluded (change < to <= to include).
3 vector<P> convexHull(vector<P> pts) {
4     int n = pts.size();
5     if (n < 2) return pts;
6     sort(pts.begin(), pts.end());
7     pts.erase(unique(pts.begin(), pts.end()), pts.end());
8     n = pts.size();
9     if (n < 2) return pts;
10
11     vector<P> hull;
12     // Lower hull
13     for (int i = 0; i < n; i++) {
14         while (hull.size() >= 2 &&
15             (hull.back() - hull[hull.size()-2]).cross(pts[i] - hull[hull.size()-2]) <= 0)
16             hull.pop_back();
17         hull.push_back(pts[i]);
18     }
19     // Upper hull
20     int lower_size = hull.size();
21     for (int i = n - 2; i >= 0; i--) {
22         while ((int)hull.size() > lower_size &&
23             (hull.back() - hull[hull.size()-2]).cross(pts[i] - hull[hull.size()-2]) <= 0)
```

```

24         hull.pop_back();
25         hull.push_back(pts[i]);
26     }
27     hull.pop_back();    // last point = first point
28     return hull;
29 }

```

Complexity

$\mathcal{O}(n \log n)$ dominated by sorting. The sweep itself is $\mathcal{O}(n)$ (each point pushed and popped at most once).

Common Trap / Gotcha

≤ 0 vs < 0 in the cross product check: Use ≤ 0 to *exclude* collinear boundary points (strict convex hull). Use < 0 to *include* them. Most problems want the strict hull.

Duplicate points: Always deduplicate before running the hull or the lower/upper size logic breaks.

12.2 Convex Hull Trick (CHT) — DP Optimization

How it Works

The **Convex Hull Trick** optimizes DP transitions of the form:

$$dp[i] = \min_{j < i} (dp[j] + b[j] \cdot x[i])$$

This is a family of **linear functions** $f_j(x) = b[j] \cdot x + dp[j]$. The minimum over all j at a query point x traces the lower envelope of these lines — which is a convex hull structure, queryable in $\mathcal{O}(\log n)$ or $\mathcal{O}(1)$ amortized.

```

1  // CHT: maintains a set of lines y = m*x + b, supports min query.
2  // Lines must be added in DECREASING slope order.
3  // Query points must be in INCREASING order (for O(1) amortized pointer).
4  struct CHT {
5      struct Line { long long m, b; };
6      deque<Line> hull;
7
8      bool bad(Line l1, Line l2, Line l3) {
9          // l2 is never the minimum: intersection of l1,l3 is left of l1,l2
10         return (__int128)(l3.b - l1.b) * (l1.m - l2.m)
11             <= (__int128)(l2.b - l1.b) * (l1.m - l3.m);
12     }
13
14     void addLine(long long m, long long b) {
15         Line l = {m, b};
16         while (hull.size() >= 2 && bad(hull[hull.size()-2], hull.back(), l))
17             hull.pop_back();
18         hull.push_back(l);
19     }
20
21     // Query minimum at x. Call with non-decreasing x for O(1) amortized.
22     long long query(long long x) {
23         while (hull.size() > 1 &&
24             hull[0].m * x + hull[0].b >= hull[1].m * x + hull[1].b)
25             hull.pop_front();
26         return hull[0].m * x + hull[0].b;
27     }
28 };

```


Key Insight / Optimization

Li Chao Tree is a segment-tree-based alternative that handles arbitrary insertion/query order in $\mathcal{O}(\log V)$ per operation (where V is the query range). Use it when slopes or queries don't arrive in sorted order.

```

1 // Li Chao Tree: supports add_line(m, b) and query_min(x) in  $\mathcal{O}(\log V)$ .
2 // For x in range [LO, HI].
3 const long long LO = -1e9, HI = 1e9, INF = 2e18;
4 struct LiChao {
5     struct Line { long long m = 0, b = INF; long long eval(long long x) { return m*x+b; } };
6     struct Node { Line line; int left = 0, right = 0; };
7     vector<Node> tree = {};
8
9     void addLine(long long m, long long b, int node, long long lo, long long hi) {
10         long long mid = lo + (hi - lo) / 2;
11         bool left_better = Line{m,b}.eval(lo) < tree[node].line.eval(lo);
12         bool mid_better = Line{m,b}.eval(mid) < tree[node].line.eval(mid);
13         if (mid_better) std::swap(tree[node].line, Line{m,b});
14         if (lo == hi) return;
15         if (left_better != mid_better) {
16             if (!tree[node].left) { tree.push_back({}); tree[node].left = tree.size()-1; }
17             addLine(m, b, tree[node].left, lo, mid); // m,b could be the swapped-out line
18         } else {
19             if (!tree[node].right) { tree.push_back({}); tree[node].right = tree.size()-1; }
20             addLine(m, b, tree[node].right, mid+1, hi);
21         }
22     }
23     void addLine(long long m, long long b) { addLine(m, b, 0, LO, HI); }
24
25     long long query(long long x, int node, long long lo, long long hi) {
26         long long res = tree[node].line.eval(x);
27         if (lo == hi) return res;
28         long long mid = lo + (hi - lo) / 2;
29         if (x <= mid && tree[node].left)
30             return min(res, query(x, tree[node].left, lo, mid));
31         if (x > mid && tree[node].right)
32             return min(res, query(x, tree[node].right, mid+1, hi));
33         return res;
34     }
35     long long query(long long x) { return query(x, 0, LO, HI); }
36 };

```

Complexity

CHT (sorted): $\mathcal{O}(n)$ total. CHT (arbitrary order): $\mathcal{O}(n \log n)$ with sorted+binary search. Li Chao Tree: $\mathcal{O}(n \log V)$ where V is the coordinate range.

12.3 Rotating Calipers — Diameter of a Convex Polygon

How it Works

The **diameter** of a point set (maximum pairwise distance) can be found in $\mathcal{O}(n \log n)$ after computing the convex hull: use **rotating calipers** to walk antipodal pairs around the hull in $\mathcal{O}(n)$.

```

1 // Diameter squared of point set (squared to avoid sqrt).  $\mathcal{O}(n \log n)$ .
2 long long diameter2(vector<P> pts) {
3     auto hull = convexHull(pts);
4     int n = hull.size();
5     if (n == 1) return 0;
6     if (n == 2) return (long long)(hull[1] - hull[0]).norm2();

```

```

7   long long best = 0;
8   int j = 1;
9   for (int i = 0; i < n; i++) {
10      P a = hull[i], b = hull[(i+1)%n];
11      while (true) {
12         P c = hull[j], d = hull[(j+1)%n];
13         // Advance j while the area of triangle increases
14         if ((b-a).cross(d-c) > 0) j = (j+1)%n;
15         else break;
16      }
17      best = max(best, (long long)(hull[j]-hull[i]).norm2());
18      best = max(best, (long long)(hull[j]-hull[(i+1)%n]).norm2());
19   }
20   return best;
21 }

```

Classic Problem

Classic: Given n points, find the two that are farthest apart.

Application: Convex hull trick in DP, furthest pair for bounding boxes, finding the width of a convex polygon (minimum distance between two parallel supporting lines).

Chapter 13

Circles & Circle Inversion

13.1 Circle Struct & Basics

How it Works

A circle is defined by a centre O and radius r . The key quantities:

- **Point inside/outside:** $|OP|^2 \leq r^2$
- **Tangent length** from external point P : $\sqrt{|OP|^2 - r^2}$
- **Circle through 3 points:** unique unless collinear (circumcircle)

```
1 struct Circle {
2     P c;      // centre
3     ld r;     // radius
4     Circle(P c, ld r) : c(c), r(r) {}
5
6     // -1 = inside, 0 = on boundary, +1 = outside
7     int contains(P p) const {
8         ld d2 = (p - c).norm2();
9         if (fabs1(d2 - r*r) < EPS) return 0;
10        return (d2 < r*r) ? -1 : 1;
11    }
12
13    // Tangent length from external point p
14    ld tangentLen(P p) const {
15        return sqrtl(max((ld)0, (p - c).norm2() - r*r));
16    }
17 };
```

13.2 Circle–Line Intersection

```
1 // Intersection of circle C with line through (a, b).
2 // Returns 0, 1, or 2 intersection points.
3 vector<P> circleLineIntersect(Circle C, P a, P b) {
4     P d = b - a;
5     P f = a - C.c;
6     ld A = d.dot(d);
7     ld B = 2 * f.dot(d);
8     ld D = f.dot(f) - C.r * C.r;
9     ld disc = B*B - 4*A*D;
10    if (disc < -EPS) return {}; // no intersection
11    disc = max((ld)0, disc);
12    ld t1 = (-B - sqrtl(disc)) / (2*A);
13    ld t2 = (-B + sqrtl(disc)) / (2*A);
14    if (fabs1(disc) < EPS) return {a + d * t1}; // tangent: one point
15    return {a + d * t1, a + d * t2};
16 }
```

13.3 Circle–Circle Intersection

```

1 // Area of intersection of two circles
2 ld circleCircleArea(Circle A, Circle B) {
3     ld d = (A.c - B.c).norm();
4     if (d >= A.r + B.r - EPS) return 0; // disjoint
5     if (d + B.r <= A.r + EPS) return acosl(-1) * B.r * B.r; // B inside A
6     if (d + A.r <= B.r + EPS) return acosl(-1) * A.r * A.r; // A inside B
7     ld a = A.r, b = B.r;
8     ld cosA = (d*d + a*a - b*b) / (2*d*a);
9     ld cosB = (d*d + b*b - a*a) / (2*d*b);
10    ld alpha = 2 * acosl(max((ld)-1, min((ld)1, cosA)));
11    ld beta = 2 * acosl(max((ld)-1, min((ld)1, cosB)));
12    return 0.5 * (a*a*(alpha - sinl(alpha)) + b*b*(beta - sinl(beta)));
13 }
14
15 // Intersection points of two circles (0, 1, or 2 points)
16 vector<P> circleCircleIntersect(Circle A, Circle B) {
17     ld d = (B.c - A.c).norm();
18     if (d < EPS || d > A.r + B.r + EPS || d < fabsl(A.r - B.r) - EPS) return {};
19     ld a = (A.r*A.r - B.r*B.r + d*d) / (2*d);
20     ld h2 = A.r*A.r - a*a;
21     if (h2 < 0) h2 = 0;
22     ld h = sqrtl(h2);
23     P mid = A.c + (B.c - A.c) * (a / d);
24     P perp = {-(B.c - A.c).y * h / d, (B.c - A.c).x * h / d};
25     if (h < EPS) return {mid};
26     return {mid + perp, mid - perp};
27 }

```

13.4 Circumcircle of 3 Points

```

1 // Circumcircle of triangle (a, b, c). Returns Circle with r=-1 if collinear.
2 Circle circumcircle(P a, P b, P c) {
3     ld ax = b.x - a.x, ay = b.y - a.y;
4     ld bx = c.x - a.x, by = c.y - a.y;
5     ld D = 2 * (ax * by - ay * bx);
6     if (fabsl(D) < EPS) return {{0,0}, -1}; // collinear
7     ld ux = (by * (ax*ax + ay*ay) - ay * (bx*bx + by*by)) / D;
8     ld uy = (ax * (bx*bx + by*by) - bx * (ax*ax + ay*ay)) / D;
9     P centre = {a.x + ux, a.y + uy};
10    return {centre, (centre - a).norm()};
11 }
12
13 // Smallest enclosing circle (Welzl's algorithm) --- O(n) expected
14 Circle welzl(vector<P>& pts, vector<P> boundary, int n);
15 Circle makeCircle(vector<P>& pts) {
16     shuffle(pts.begin(), pts.end(), mt19937(42));
17     return welzl(pts, {}, pts.size());
18 }
19 Circle welzl(vector<P>& pts, vector<P> R, int n) {
20     if (n == 0 || R.size() == 3) {
21         if (R.empty()) return {{0,0}, 0};
22         if (R.size() == 1) return {R[0], 0};
23         if (R.size() == 2) return {(R[0]+R[1])*(0.5), (R[0]-R[1]).norm()/2};
24         return circumcircle(R[0], R[1], R[2]);
25     }
26     Circle D = welzl(pts, R, n - 1);
27     if (D.contains(pts[n-1]) <= 0) return D;
28     R.push_back(pts[n-1]);
29     return welzl(pts, R, n - 1);
30 }

```

Complexity

Circumcircle: $\mathcal{O}(1)$. Smallest enclosing circle (Welzl): $\mathcal{O}(n)$ expected.

13.5 Circle Inversion

How it Works

Circle Inversion with respect to a circle ω (centre O , radius r) maps every point $P \neq O$ to a point P' on ray $\overrightarrow{OP}$ such that:

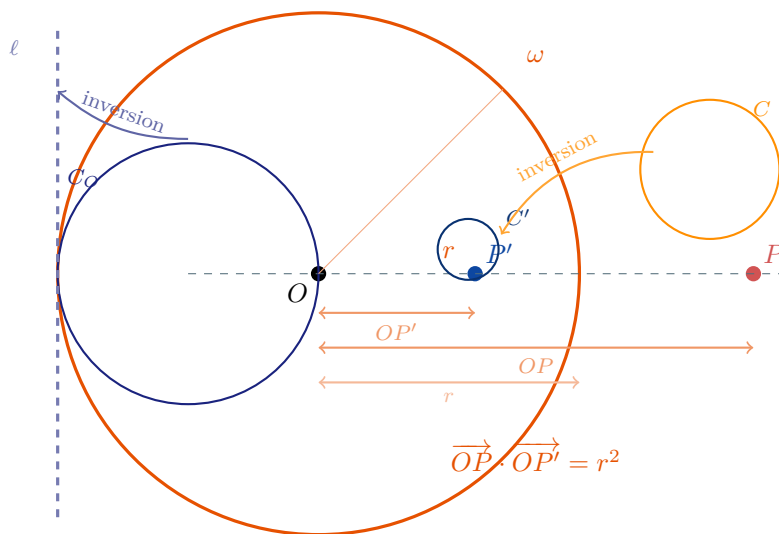
$$\overrightarrow{OP} \cdot \overrightarrow{OP'} = r^2 \quad \Longleftrightarrow \quad P' = O + \frac{r^2}{|OP|^2}(P - O)$$

Key mapping rules under inversion:

- **Circle not through $O \rightarrow$ circle not through O**
- **Circle through $O \rightarrow$ line (not through O)**
- **Line not through $O \rightarrow$ circle through O**
- **Line through $O \rightarrow$ itself**

Inversion **preserves angles** (it is a conformal map) and **preserves the inversion circle itself**.

Diagram



```

1 // Invert point P with respect to circle omega (centre O, radius r)
2 P invertPoint(P P_, P O, ld r) {
3     P d = P_ - O;
4     ld norm2 = d.norm2();
5     if (norm2 < EPS) return P_; // O itself is undefined; return as-is
6     return O + d * (r * r / norm2);
7 }
8
9 // Invert a circle C w.r.t. inversion circle omega.
10 // Returns the image as a Circle (may be a line --- represented by r = INF).
11 // For CP purposes we handle the two cases:
12 // (a) C does not pass through O --> circle
13 // (b) C passes through O --> line (returned as a circle with huge r)
14 Circle invertCircle(Circle C, P O, ld r) {
15     ld d = (C.c - O).norm();
16
17     if (fabs1(d - C.r) < EPS) {
18         // C passes through O: image is a line

```

```

19 // Direction perpendicular to OC, through invertPoint of the far point
20 P far = O + (C.c - O) * ((d + C.r) / d); // far intersection with OC
21 P img = invertPoint(far, O, r);
22 // Return a "line circle" centred very far away (caller must handle)
23 // In practice: return the centre and a flag; here we use r = 1e18
24 return {img, (ld)1e18};
25 }
26
27 // General case: C does not pass through O
28 // New centre and radius via the inversion formula
29 ld k = r * r;
30 ld d2 = d * d - C.r * C.r;
31 ld new_r = fabs1(k * C.r / d2);
32 P new_c = O + (C.c - O) * (k / d2);
33 return {new_c, new_r};
34 }

```

Key Insight / Optimization

Why inversion is useful in CP: Hard problems involving many tangent circles (e.g. Apollonius circles, Soddy's theorem) become trivial after inversion — tangent circles map to tangent circles or parallel lines, turning curved configurations into rectilinear ones. The classic trick is to invert at a point of tangency to straighten a cluster of mutually tangent circles into a simpler arrangement.

Common Trap / Gotcha

O has no image. The centre of inversion is sent to infinity. Always check that the points you are inverting are not O itself. Additionally, the formula $OP \cdot OP' = r^2$ is a *signed* statement along the ray — negative values would map to the opposite ray, which doesn't occur for real points but can arise numerically.

Chapter 14

Advanced Geometry

14.1 Polar Angle Sort

How it Works

Sorting points by the angle they make with a reference direction (usually the positive x -axis from some origin). Essential for angular sweep algorithms, half-plane intersection, and rotating calipers variants.

```
1 // Half-plane of a point: 0 = lower (y < 0 or (y==0 and x<0)), 1 = upper
2 int half(P p) { return p.y < -EPS || (fabs1(p.y) < EPS && p.x < -EPS) ? 0 : 1; }
3
4 // Comparator: sort by polar angle in [0, 2pi). Ties broken by distance.
5 // Origin is treated as the reference; shift all points by origin first.
6 bool polarCmp(P a, P b) {
7     int ha = half(a), hb = half(b);
8     if (ha != hb) return ha < hb;
9     ld cross = a.cross(b);
10    if (fabs1(cross) > EPS) return cross > 0; // a is CCW from b
11    return a.norm2() < b.norm2(); // same angle: closer first
12 }
13
14 // Usage: sort points relative to origin O
15 void polarSort(vector<P>& pts, P O) {
16     for (auto& p : pts) p = p - O;
17     sort(pts.begin(), pts.end(), polarCmp);
18     for (auto& p : pts) p = p + O;
19 }
```

14.2 Half-Plane Intersection

How it Works

A **half-plane** is the set of points on one side of a directed line. The intersection of n half-planes is a convex region (possibly unbounded or empty).

Algorithm (Sorted Half-Plane Intersection):

1. Sort half-planes by angle (direction of their boundary).
2. Process in sorted order, maintaining a deque of active half-planes.
3. Each new half-plane may invalidate the back of the deque (intersection point no longer satisfies the constraint).
4. Final pass to clean the front of the deque against the back.

Runs in $\mathcal{O}(n \log n)$ dominated by sorting.

```
1 struct HalfPlane {
```

```

2   P p, d;    // points on the left side of the ray p -> p+d
3   ld angle;
4   HalfPlane(P p, P d) : p(p), d(d), angle(atan2l(d.y, d.x)) {}
5   // Is point q inside this half-plane?
6   bool inside(P q) const { return d.cross(q - p) > -EPS; }
7 };
8
9 P hpIntersect(const HalfPlane& a, const HalfPlane& b) {
10     ld t = (b.d).cross(a.p - b.p) / (a.d).cross(b.d);
11     return a.p + a.d * t;
12 }
13
14 // Returns vertices of the half-plane intersection polygon.
15 // Empty = infeasible (all planes contradictory), unbounded = open region.
16 vector<P> halfPlaneIntersection(vector<HalfPlane> hps) {
17     sort(hps.begin(), hps.end(),
18         [](const HalfPlane& a, const HalfPlane& b) {
19             if (fabsl(a.angle - b.angle) > EPS) return a.angle < b.angle;
20             return b.inside(a.p); // keep the more restrictive one
21         });
22     // Remove duplicate-angle planes (keep only the most restrictive)
23     hps.erase(unique(hps.begin(), hps.end(),
24         [](const HalfPlane& a, const HalfPlane& b) {
25             return fabsl(a.angle - b.angle) < EPS;
26         }), hps.end());
27
28     int n = hps.size();
29     deque<HalfPlane> dq;
30     deque<P> pts;
31
32     for (int i = 0; i < n; i++) {
33         while (pts.size() && !hps[i].inside(pts.back())) {
34             pts.pop_back(); dq.pop_back();
35         }
36         while (pts.size() && !hps[i].inside(pts.front())) {
37             pts.pop_front(); dq.pop_front();
38         }
39         dq.push_back(hps[i]);
40         if (dq.size() >= 2)
41             pts.push_back(hpIntersect(dq[dq.size()-2], dq.back()));
42     }
43     // Final cleanup
44     while (pts.size() && !dq.front().inside(pts.back())) {
45         pts.pop_back(); dq.pop_back();
46     }
47     if (dq.size() < 3) return {};
48     pts.push_back(hpIntersect(dq.back(), dq.front()));
49     return {pts.begin(), pts.end()};
50 }

```

Complexity

$\mathcal{O}(n \log n)$. The intersection polygon has at most n vertices.

Classic Problem

Classic: Given a convex polygon and a point P , find the maximum area sub-polygon visible from P — equivalent to intersecting n half-planes defined by the polygon's edges (from P 's perspective) with the polygon's half-planes.

Classic 2: Find the region equidistant from a set of points (Voronoi cell of a point = intersection of $n - 1$ half-planes).

14.3 Closest Pair of Points

How it Works

Brute force is $\mathcal{O}(n^2)$. Divide-and-conquer finds the closest pair in $\mathcal{O}(n \log n)$ by:

1. Sort by x . Split into left/right halves.
2. Recursively find $\delta = \min(\delta_L, \delta_R)$.
3. Only check cross-boundary pairs within a 2δ -wide strip.
4. Key insight: within the strip, each point has at most **7 neighbours** to check.

```

1  using ll = long long;
2
3  // Squared distance (integer, exact)
4  ll dist2(P a, P b) {
5      ll dx = a.x - b.x, dy = a.y - b.y;
6      return dx*dx + dy*dy;
7  }
8
9  ll closestPairRec(vector<P>& pts, int l, int r) {
10     if (r - l <= 3) {
11         ll best = LLONG_MAX;
12         for (int i = l; i < r; i++)
13             for (int j = i+1; j < r; j++)
14                 best = min(best, dist2(pts[i], pts[j]));
15         sort(pts.begin()+l, pts.begin()+r, [](P a, P b){ return a.y < b.y; });
16         return best;
17     }
18     int mid = (l + r) / 2;
19     ll midX = pts[mid].x;
20     ll d = min(closestPairRec(pts, l, mid), closestPairRec(pts, mid, r));
21
22     // Merge by y
23     inplace_merge(pts.begin()+l, pts.begin()+mid, pts.begin()+r,
24                  [](P a, P b){ return a.y < b.y; });
25
26     // Strip: only points within sqrt(d) of the midline
27     vector<P> strip;
28     for (int i = l; i < r; i++)
29         if ((pts[i].x - midX)*(pts[i].x - midX) < d)
30             strip.push_back(pts[i]);
31
32     // Each point checks at most 7 ahead
33     for (int i = 0; i < (int)strip.size(); i++)
34         for (int j = i+1; j < (int)strip.size()
35              && (strip[j].y-strip[i].y)*(strip[j].y-strip[i].y) < d; j++)
36             d = min(d, dist2(strip[i], strip[j]));
37     return d;
38 }
39
40 // Returns squared distance of closest pair.  $\mathcal{O}(n \log n)$ .
41 ll closestPair(vector<P> pts) {
42     sort(pts.begin(), pts.end());
43     return closestPairRec(pts, 0, pts.size());
44 }

```

Complexity

$\mathcal{O}(n \log n)$. The “at most 7 neighbours” bound comes from a packing argument: in a $2\delta \times \delta$ rectangle, you can fit at most 8 points all mutually at distance $\geq \delta$.

14.4 Minkowski Sum of Convex Polygons

How it Works

The **Minkowski sum** $A \oplus B = \{a + b \mid a \in A, b \in B\}$ of two convex polygons with m and n vertices is a convex polygon with at most $m + n$ vertices. Applications: collision detection, checking if one polygon can be translated to intersect another.

Algorithm: Start from the lowest points of each polygon. Merge their edge vectors sorted by angle (like merging two sorted lists).

```

1 // Minkowski sum of two CCW convex polygons A and B. O(m + n).
2 vector<P> minkowski(vector<P> A, vector<P> B) {
3     // Rotate both so they start at the bottom-most point
4     auto bot = [] (vector<P>& P_) {
5         int idx = min_element(P_.begin(), P_.end()) - P_.begin();
6         rotate(P_.begin(), P_.begin()+idx, P_.end());
7     };
8     bot(A); bot(B);
9     A.push_back(A[0]); A.push_back(A[1]); // wrap for edge access
10    B.push_back(B[0]); B.push_back(B[1]);
11
12    vector<P> result;
13    int i = 0, j = 0;
14    int na = A.size()-2, nb = B.size()-2;
15    while (i < na || j < nb) {
16        result.push_back(A[i] + B[j]);
17        P ea = A[i+1] - A[i];
18        P eb = B[j+1] - B[j];
19        ld cross = ea.cross(eb);
20        if (cross > EPS) { i++; }
21        else if (cross < -EPS) { j++; }
22        else { i++; j++; }
23    }
24    return result;
25 }
```

Classic Problem

Segment reachability: Can a convex robot B (as a polygon) translate from position s to t without hitting a convex obstacle A ? Answer: check if segment $[s, t]$ intersects $A \oplus (-B)$ (the Minkowski sum with the reflection of B).